

《机器学习基础》



编码器-解码器架构

https://github.com/styluck/mech_learn

内容

▶ 为什么需要“表示学习”

▶ 稀疏编码

▶ 编码器-解码器架构

▶ 自编码器

▶ 稀疏自编码器

▶ 降噪自编码器

▶ U-Net

▶ 概率密度估计

▶ 参数密度估计

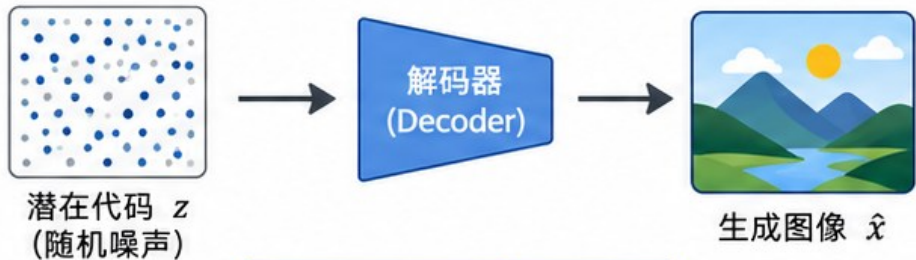
▶ 非参数密度估计

▶ GAN

从数据表示到数据重构

1

图像生成

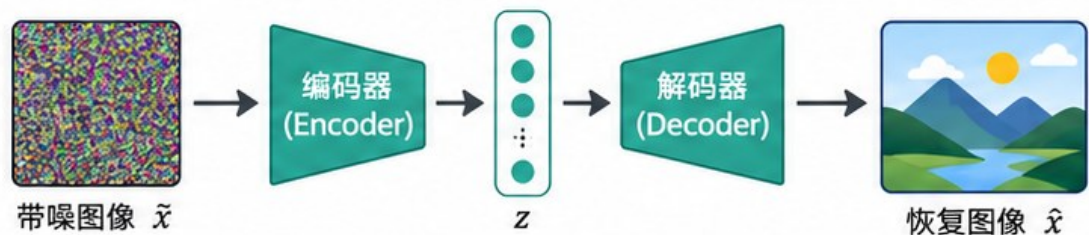


$$z \rightarrow \hat{x}$$

从潜在表示出发，生成新的图像样本。

2

图像去噪

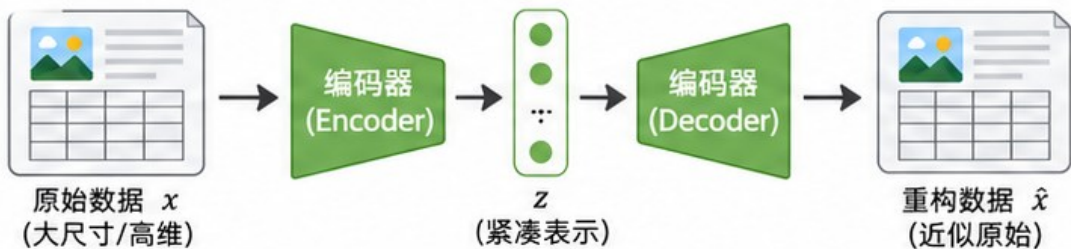


$$\tilde{x} \rightarrow z \rightarrow \hat{x} \approx x$$

从带噪输入中恢复更干净的图像。

3

数据压缩

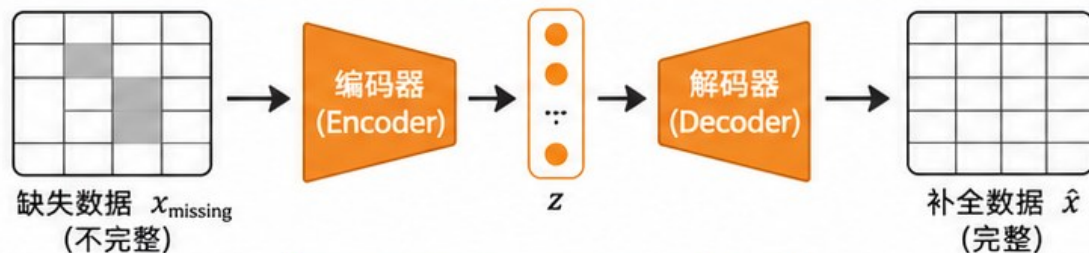


$$x \rightarrow z \rightarrow \hat{x}$$

用低维表示保留主要信息，减少存储开销。

4

缺失数据补全



$$x_{\text{missing}} \rightarrow z \rightarrow \hat{x}$$

根据数据结构推断并补全缺失部分。

为什么需要“表示学习”

▶ 监督学习

- ▶ 建立映射关系 $f: x \rightarrow y$

▶ 无监督学习

- ▶ 指从无标签的数据中学习出一些有用的模式：

$$x \rightarrow z$$

- ▶ 其中 z 是数据的潜在表示。
- ▶ 聚类：根据样本之间的相似性，将样本自动划分为若干组
 - ▶ 不借助于任何人工给出标签或者反馈等指导信息
- ▶ 特征学习：学习有用的潜在中间表示

回顾主成分分析(PCA)

► x_i : 原始数据, y_i : PCA 降到 k 维后的数据, $\hat{x}_i$: 由 y_i 重构回原空间后的数据。

► 平均投影误差:

$$\frac{1}{n} \sum_{i=1}^n ||x_i - \hat{x}_i||^2$$

► PCA 的思想是: 用少数几个正交主成分表示数据。

► 稀疏编码的思想是: 用一个可能过完备的字典表示数据, 但只激活其中少数几个基向量。

(线性) 编码

- ▶ 给定一组基向量 $A = [\mathbf{a}_1, \dots, \mathbf{a}_M]$, 将输入样本 $\mathbf{x}$ 表示为这些基向量的线性组合

$$\mathbf{x} = \sum_{m=1}^M z_m \mathbf{a}_m$$
$$= \mathbf{A}\mathbf{z},$$

字典 (dictionary)

编码 (encoding)

$$\mathbf{x} = \mathbf{A}\mathbf{z}$$

- ▶ 维度: $A \in R^{D \times M}$, $z \in R^M$, $x \in R^D$
- ▶ 当 $M > D$ 时, 字典向量个数多于原始维度, 称为过完备字典。

过完备表示

数学小知识 | 完备性

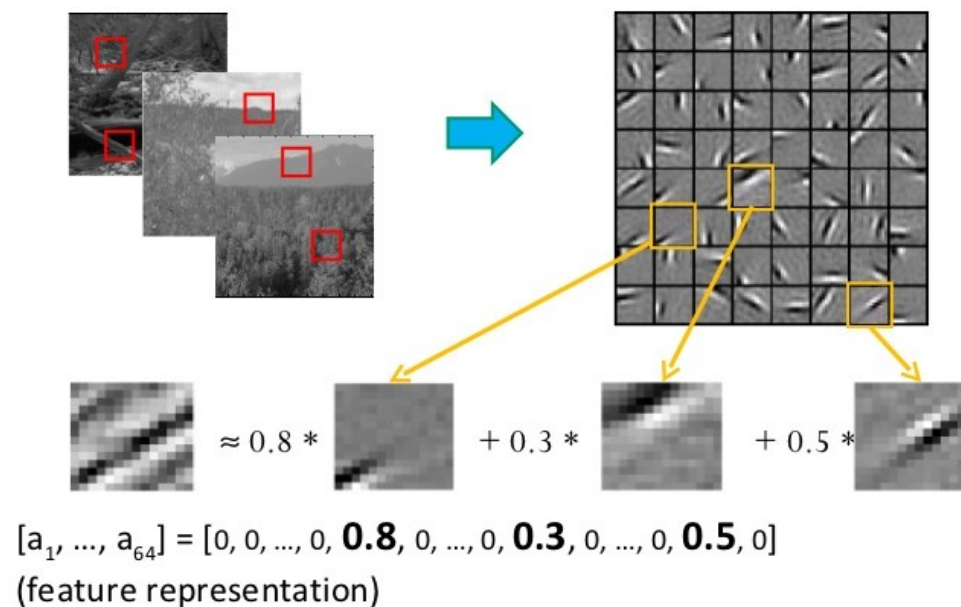
如果 M 个基向量刚好可以支撑 M 维的欧氏空间, 则这 M 个基向量是完备的. 如果 M 个基向量可以支撑 D 维的欧氏空间, 并且 $M > D$, 则这 M 个基向量是过完备的 (overcomplete)、冗余的.

“过完备”基向量是指基向量个数远远大于其支撑空间维度. 因此这些基向量一般不具备独立、正交等性质.

► 稀疏编码

► 找到一组“过完备”的基向量
(即 $M > D$) 来进行编码。

Sparse coding illustration



Slide credit: Andrew Ng

Compact & easily interpretable

稀疏编码 (Sparse Coding)

▶ 给定一组 N 个输入向量 $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}$ ，其稀疏编码的目标函数定义为

$$\mathcal{L}(\mathbf{A}, \mathbf{Z}) = \sum_{n=1}^N \left(\left\| \mathbf{x}^{(n)} - \mathbf{A}\mathbf{z}^{(n)} \right\|^2 + \eta \rho(\mathbf{z}^{(n)}) \right) + \lambda \frac{1}{2} \|\mathbf{A}\|^2$$

▶ $\rho(\cdot)$ 是一个稀疏性衡量函数， η 是一个超参数，用来控制稀疏性的强度。

$$\rho(\mathbf{z}) = \sum_{i=1}^p \mathbf{I}(|z_i| > 0)$$

$$\rho(\mathbf{z}) = \sum_{i=1}^p -\exp(-z_i^2)$$

$$\rho(\mathbf{z}) = \sum_{i=1}^p |z_i|$$

$$\rho(\mathbf{z}) = \sum_{i=1}^p \log(1 + z_i^2)$$

训练过程

► 稀疏编码的训练过程一般用交替优化的方法进行。

1) 固定基向量 $\mathbf{A}$, 对每个输入 $\mathbf{x}^{(n)}$, 计算其对应的最优编码

$$\min_{\mathbf{z}^{(n)}} \left\| \mathbf{x}^{(n)} - \mathbf{A}\mathbf{z}^{(n)} \right\|^2 + \eta \rho(\mathbf{z}^{(n)}), \quad \forall n \in [1, N].$$

2) 固定上一步得到的编码 $\{\mathbf{z}^{(n)}\}_{n=1}^N$, 计算其最优的基向量

$$\min_{\mathbf{A}} \sum_{n=1}^N \left(\left\| \mathbf{x}^{(n)} - \mathbf{A}\mathbf{z}^{(n)} \right\|^2 \right) + \lambda \frac{1}{2} \|\mathbf{A}\|^2,$$

稀疏编码的优点

► 计算量

- 稀疏编码的表示紧凑、可解释性强、具有一定特征选择作用。
- 本身通常要求解优化问题，训练和编码成本并不一定低。

► 可解释性

- 因为稀疏编码只有少数的非零元素，相当于将一个输入样本表示为少数几个相关的特征。这样其特征更易于解释。

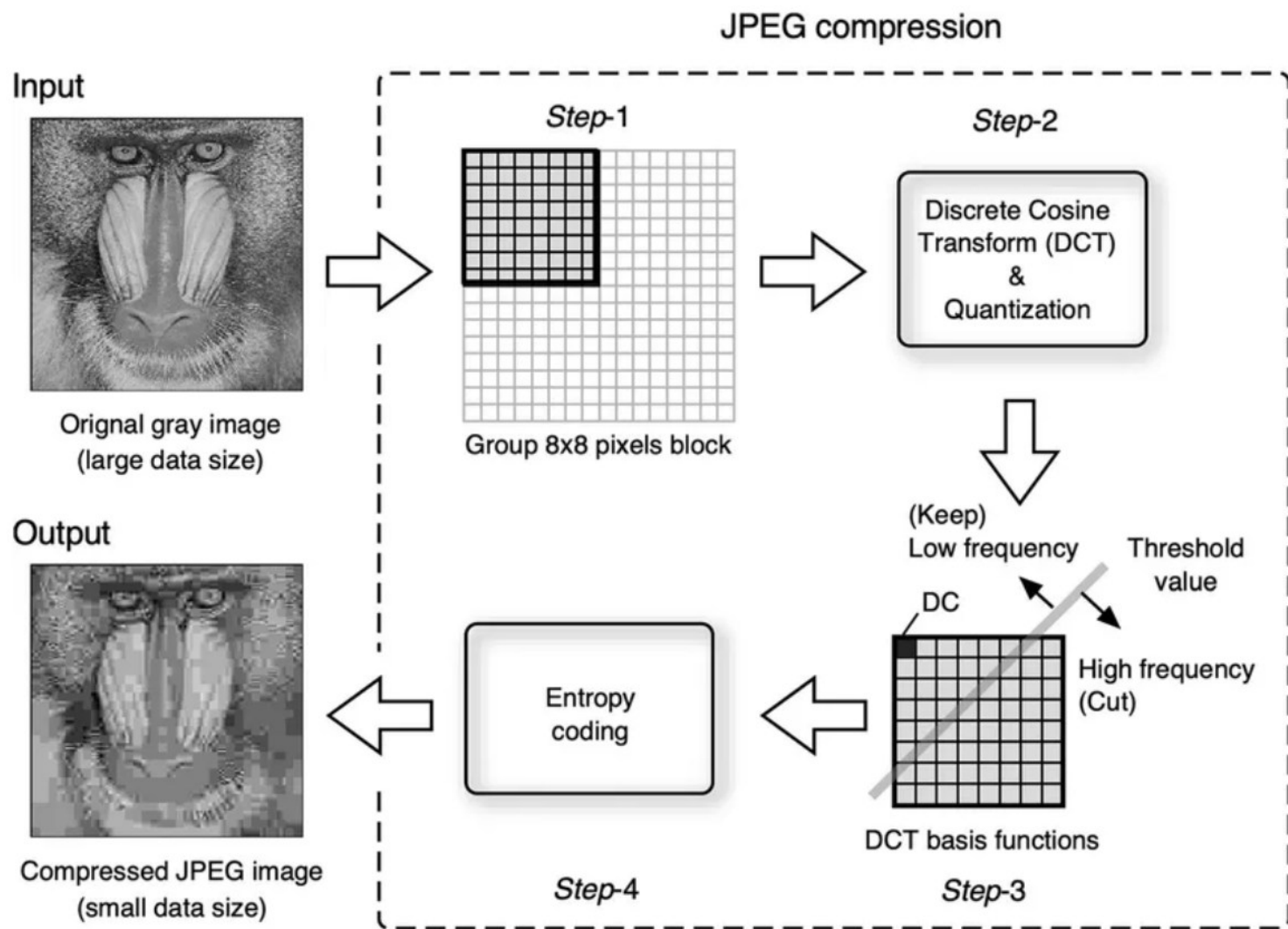
► 特征选择

- 稀疏性可助于实现特征的自动选择，只选择和输入样本相关的最少特征，在下游任务（如解码）时可以更好地表示输入样本，降低噪声并减轻过拟合。



自编码器

人工设计的编码-解码流程



JPEG pipeline (Figure is taken from[3])

从稀疏编码到自编码器

► 稀疏编码的表示形式是：

$$x \approx Az$$

► 其中 z 是编码， A 是字典。

► 对每一个新的样本 x ，都需要重新求解一个优化问题来得到 z 。

自编码器

▶ 自编码器:

- ▶ 找一个编码器 $z = f_{\theta}(x)$, 让 f_{θ} 快速给出编码表示 z ;
- ▶ 再用解码器 $g_{\phi}(z)$ 将编码表示 z 重构为输入 $\hat{x}$ 。
- ▶ 编码器 (Encoder) 加工输入
- ▶ 解码器 (Decoder) 生成输出



- ▶ 通过编码 - 解码过程实现输入数据的 “重构”，并在这一过程中学习数据的潜在特征表示

自编码器 (Auto-Encoder)

► 编码器 (Encoder)

► 解码器 (Decoder)

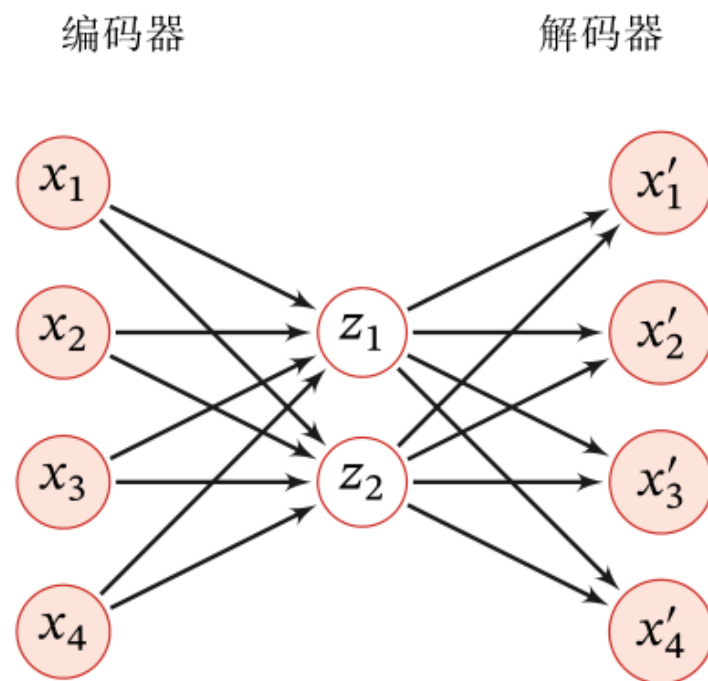
$$f : \mathbb{R}^D \rightarrow \mathbb{R}^M$$

$$g : \mathbb{R}^M \rightarrow \mathbb{R}^D$$

► 目标函数：重构错误

$$\begin{aligned}\mathcal{L} &= \sum_{n=1}^N \|\mathbf{x}^{(n)} - g(f(\mathbf{x}^{(n)}))\|^2 \\ &= \sum_{n=1}^N \|\mathbf{x}^{(n)} - f \circ g(\mathbf{x}^{(n)})\|^2.\end{aligned}$$

► 两层网络结构的自编码器



稀疏自编码器

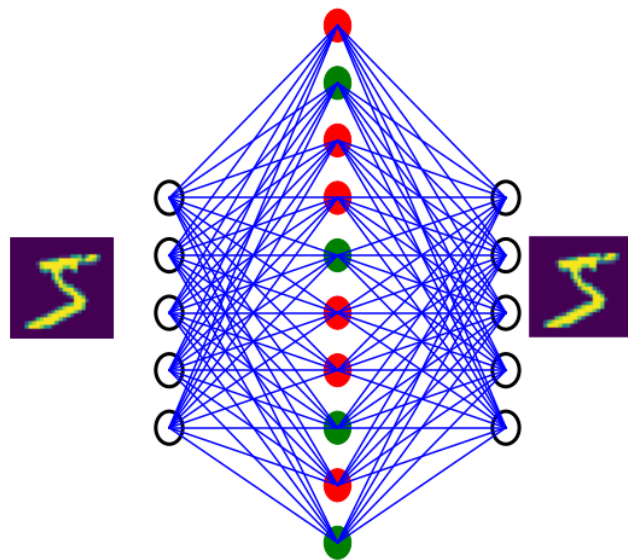
▶ 通过给自编码器中隐藏层单元 z 加上稀疏性限制，自编码器可以学习到数据中一些有用的结构。

▶ 目标函数

$$\min_{\theta} \sum_{n=1}^N ||x^{(n)} - x'^{(n)}||^2 + \eta \rho(z) + \lambda ||\theta||^2$$

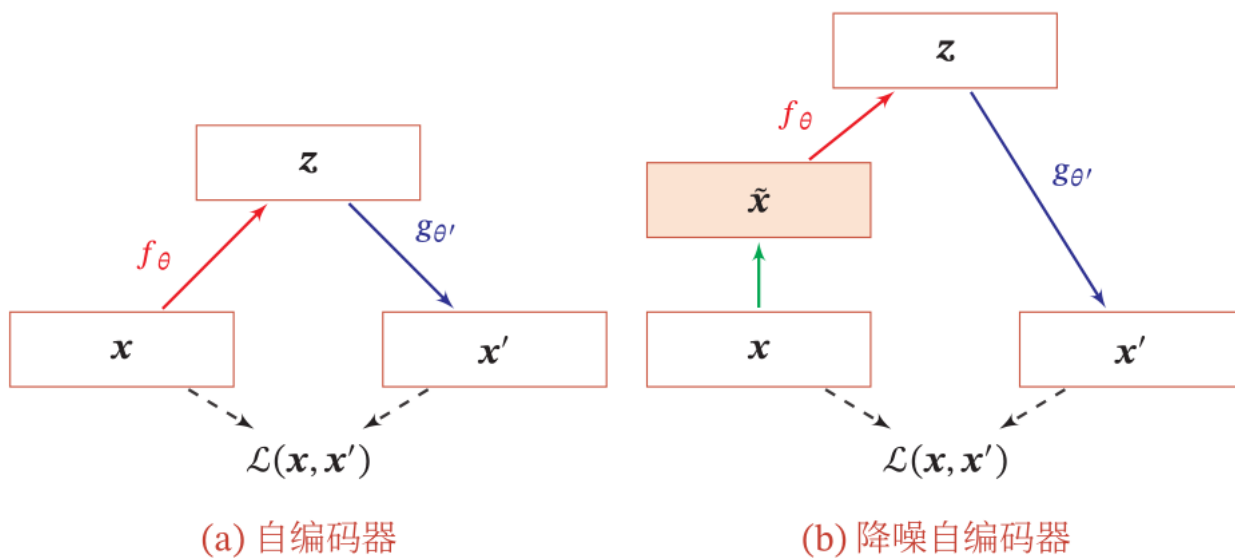
▶ θ 表示自编码器中的参数

▶ 和稀疏编码一样，稀疏自编码器的优点是有很高的可解释性，并同时进行了隐式的特征选择。



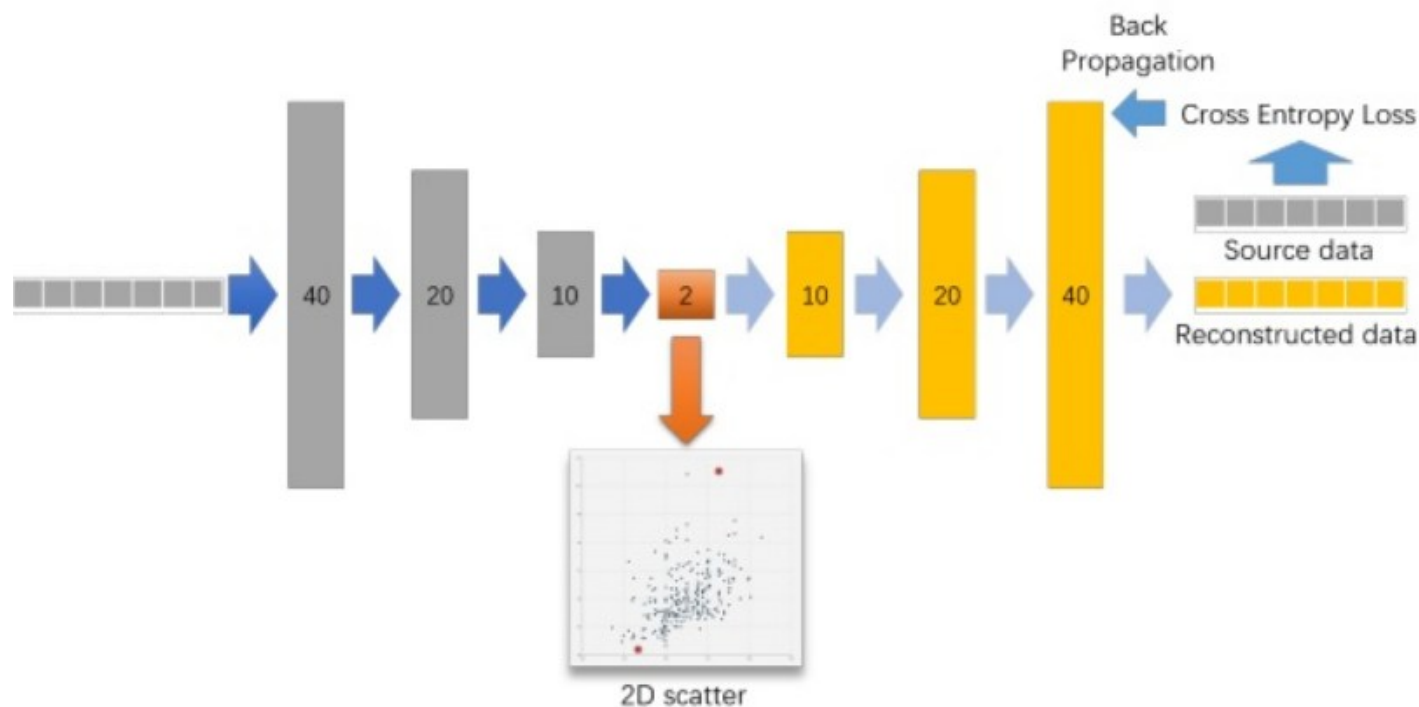
降噪自编码器

- ▶ 通过引入噪声来增加编码鲁棒性的自编码器，例如：
 - ▶ 随机置零：对于一个向量 $\mathbf{x}$ ，根据比例 μ 随机将 $\mathbf{x}$ 的一些元素设为 0，得到一个被损坏的向量 $\tilde{\mathbf{x}}$ 。
 - ▶ 加高斯噪声： $\tilde{\mathbf{x}} = \mathbf{x} + \epsilon$ ， $\epsilon \sim N(0, 1)$
 - ▶ 训标是从损坏输入 $\tilde{\mathbf{x}}$ 重构原始输入 $\mathbf{x}$ ： $\min_{\theta, \phi} \sum_i \|\mathbf{x}_i - g_{\phi}(f_{\theta}(\tilde{\mathbf{x}}_i))\|^2$ 。



堆叠自编码器

- ▶ 仅使用两层神经网络的自编码器还不足以获取一种好的数据表示。
- ▶ 为了获取更好的数据表示，我们可以使用更深层的神经网络。



Encoder-Decoder 的局限

- ▶ 编码器-解码器结构会先压缩信息，再恢复输出。
 - ▶ 对于分类任务，这种压缩是可以接受的；
 - ▶ 但对于图像任务，输出需要保留精确的空间位置。
- ▶ 普通 encoder-decoder 有一个矛盾：
 - ▶ 编码器需要提取高层语义，但下采样会损失空间细节；
 - ▶ 解码器需要恢复空间分辨率，但仅靠低维表示很难恢复边界细节。

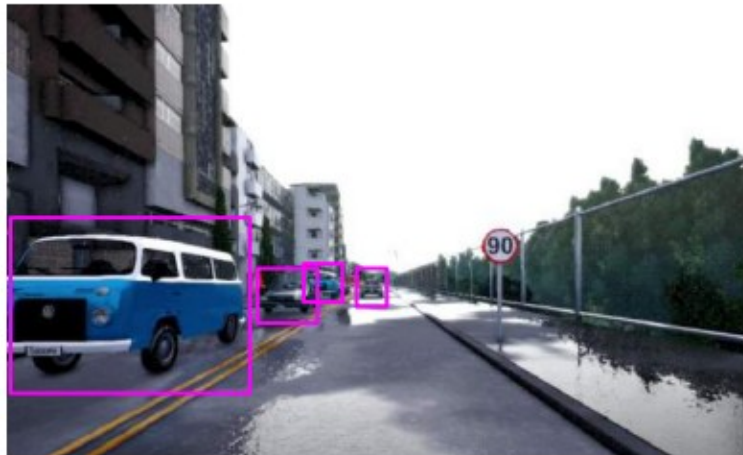


U-Net

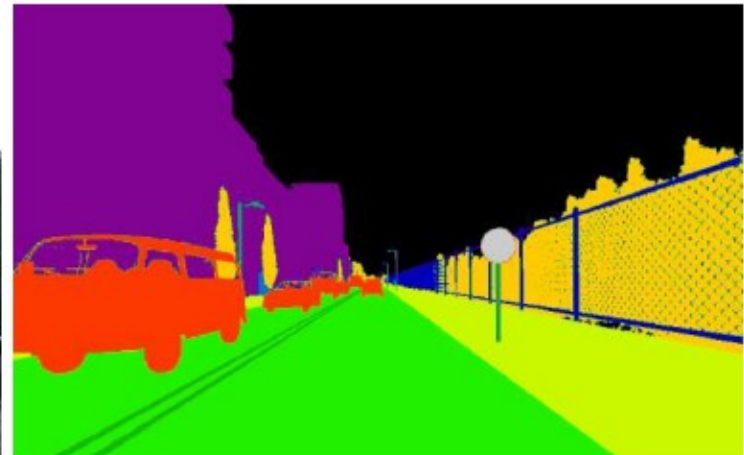
图像分割任务



Input image



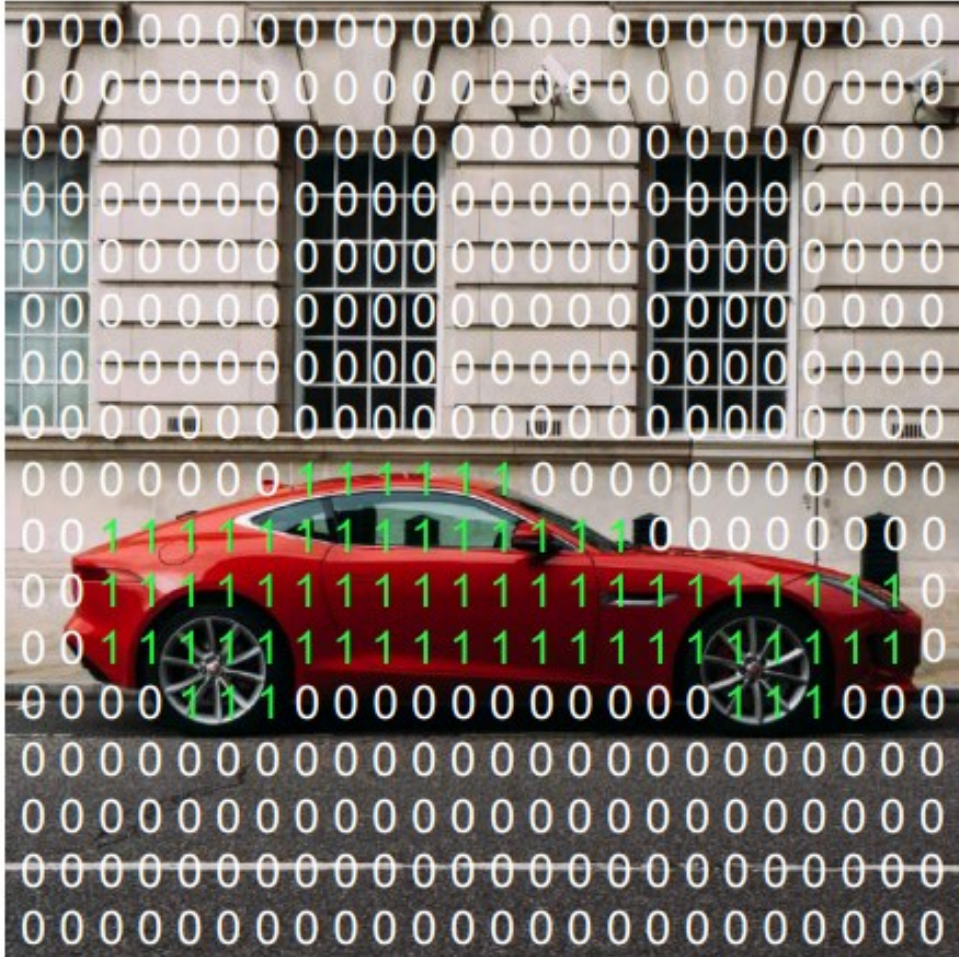
Object Detection



Semantic Segmentation

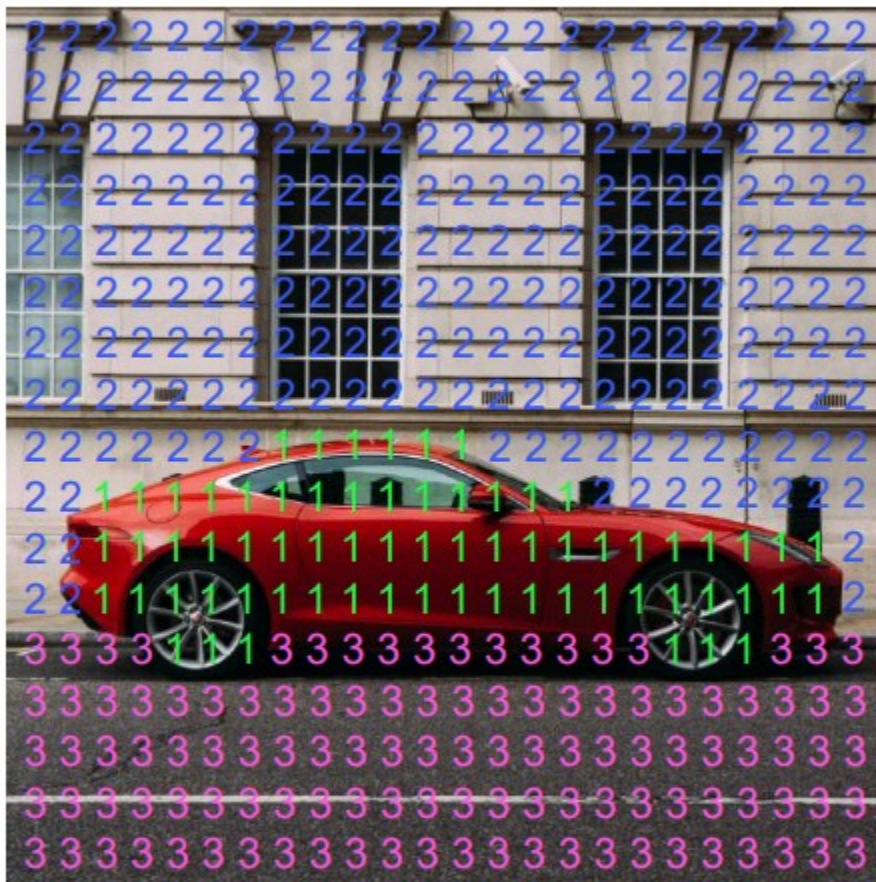
<https://cs230.stanford.edu/files/C4M3.pdf>

Per-pixel class labels



- ▶ 1. Car
- ▶ 0. Not Car

Per-pixel class labels



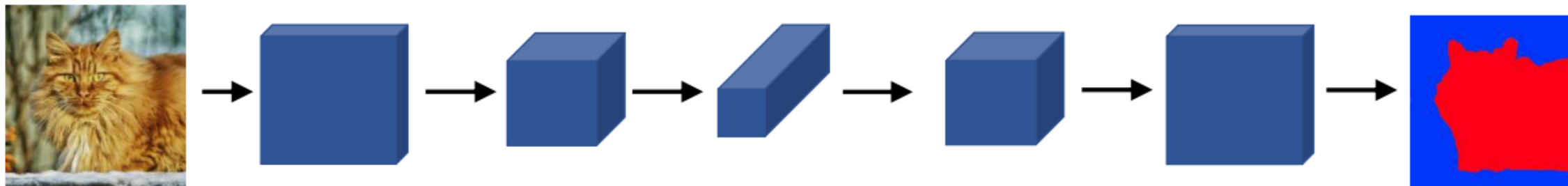
1. Car
2. Building
3. Road



图像分割学习

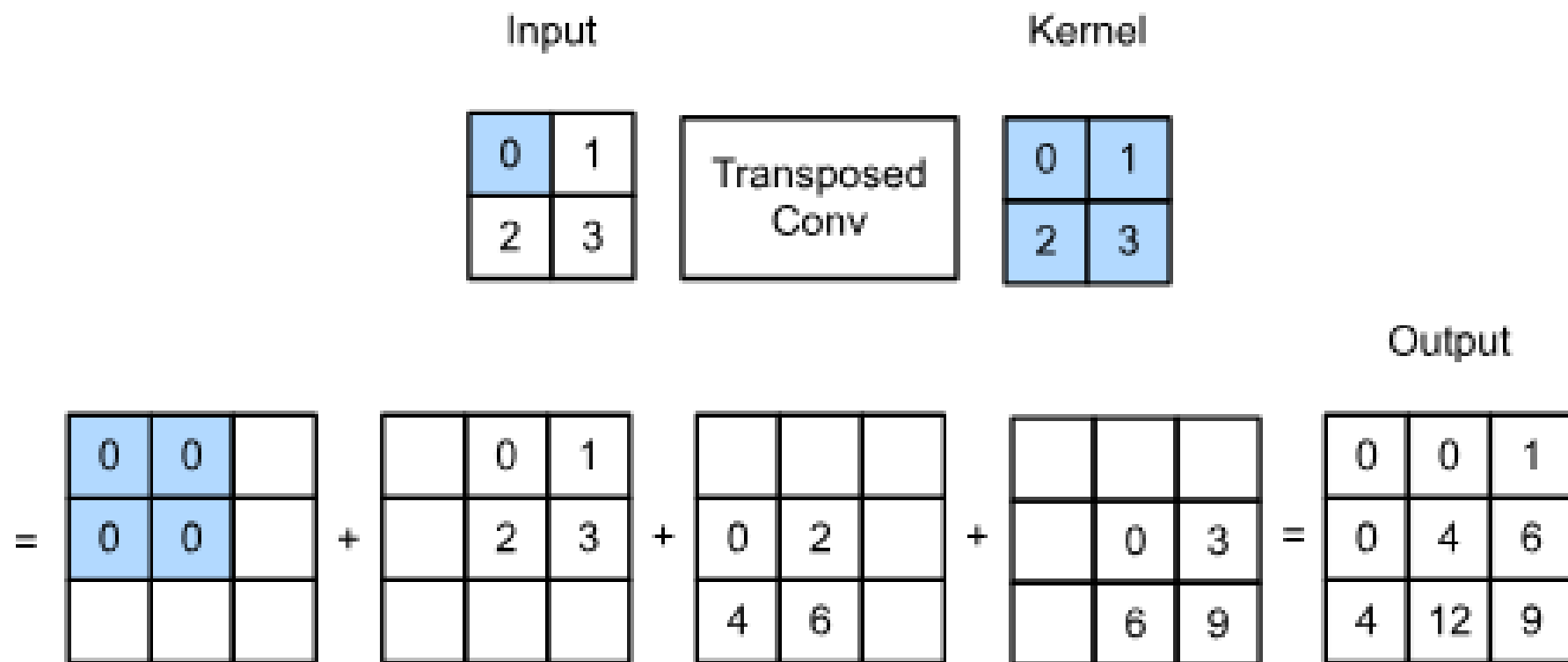
► 图像分割 = 像素级分类

- 输入图像经过卷积网络提取特征后，
- 输出每个像素所属的类别，形成分割掩膜。



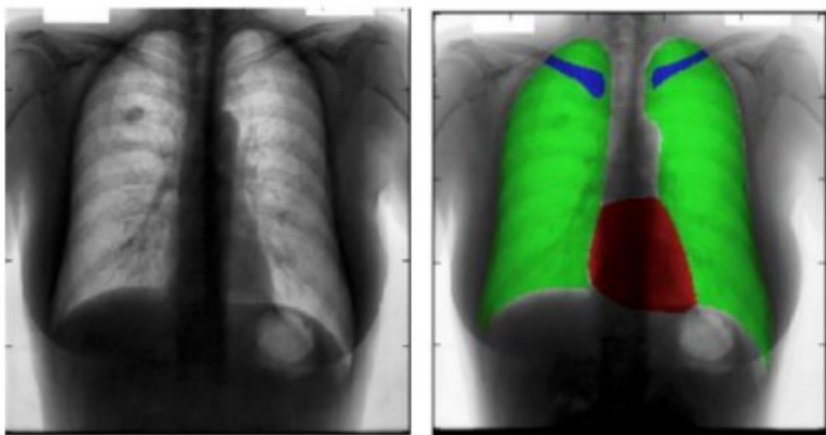
- 如何将小的特征图恢复为大的特征图？

转置卷积

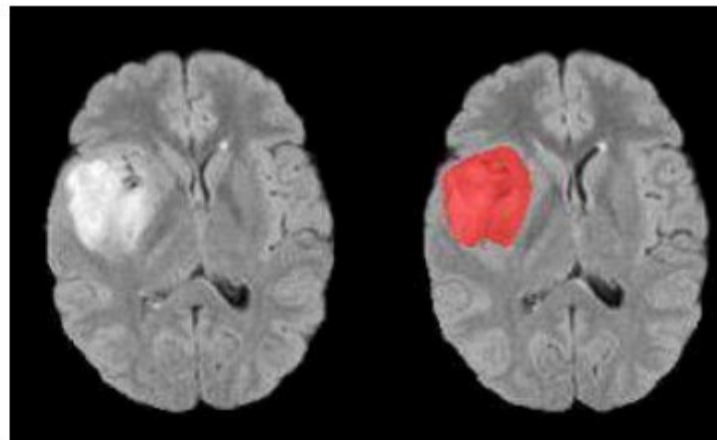


U-Net

- ▶ **U-Net: 特殊设计的 Encoder-Decoder 结构。**
 - ▶ 在原始论文中，它主要用于监督式医学图像分割，
 - ▶ 训练时需要输入图像及其对应的像素级分割标注。
 - ▶ 普通 Encoder-Decoder 网络可以完成图像分割，但在下采样过程中会损失空间细节，导致边界恢复不够精确。

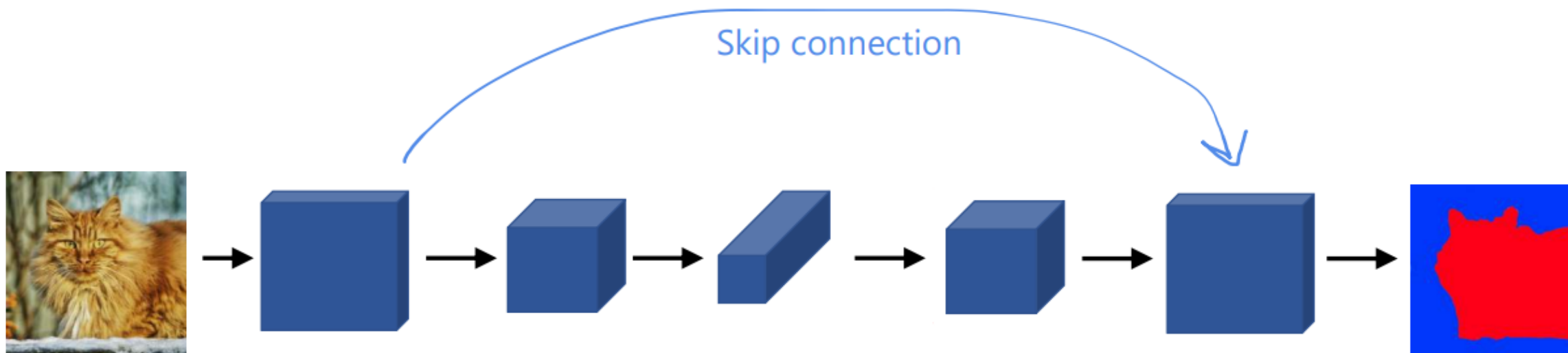


Chest X-Ray



Brain MRI

图像分割学习



Skip Connection

▶ 直接连接编码器和解码器的对应层：

▶ 将编码器中浅层的高分辨率特征（含空间细节）与解码器中同尺寸的特征图（含语义信息）融合，形成“语义+细节”的互补特征。

▶ 若解码器某层特征为 D_i ，编码器对应层特征为 E_i ，则拼接后特征

$$F_i = \text{Concat}(D_i, E_i)$$

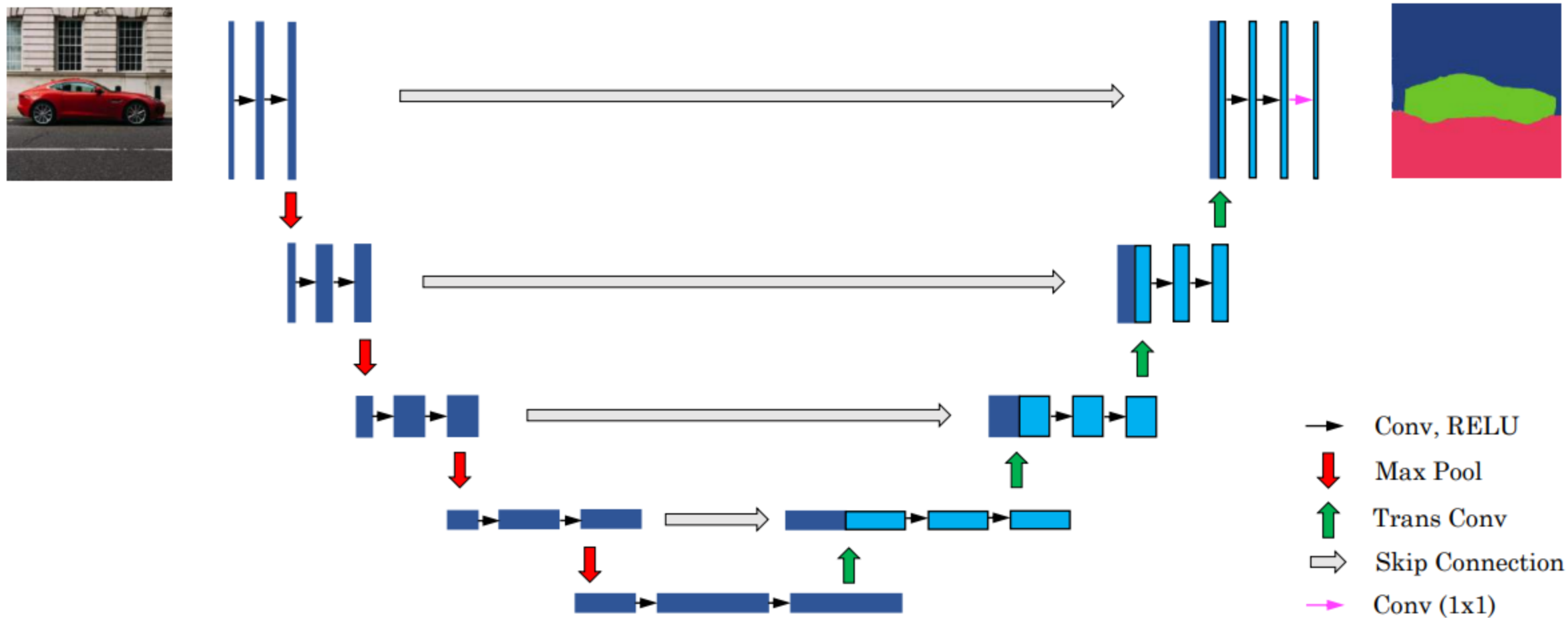
▶ 拼接前 D_i 和 E_i 的空间尺寸需要一致，

▶ Concat 通常发生在通道维度上。

Skip Connection

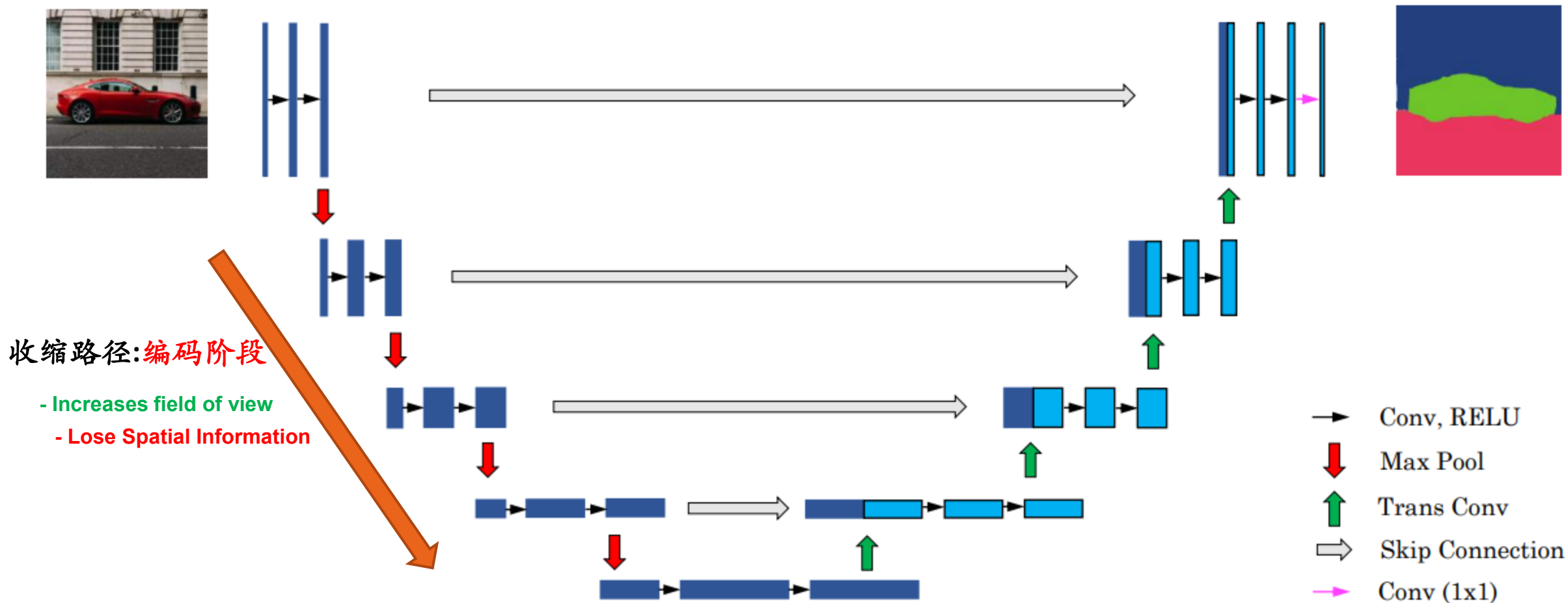
- ▶ 图像分割需要判断每一个像素属于什么类别：
 - ▶ 要知道图中这是什么，也要知道这在图中哪里
 - ▶ 传统编码器后面深层特征知道图中是什么，但不太清楚精确边界在哪里
- ▶ 跳跃连接就是为了解决这个问题
 - ▶ 高层语义+低层细节：更准确的像素级分割

U-Net 架构



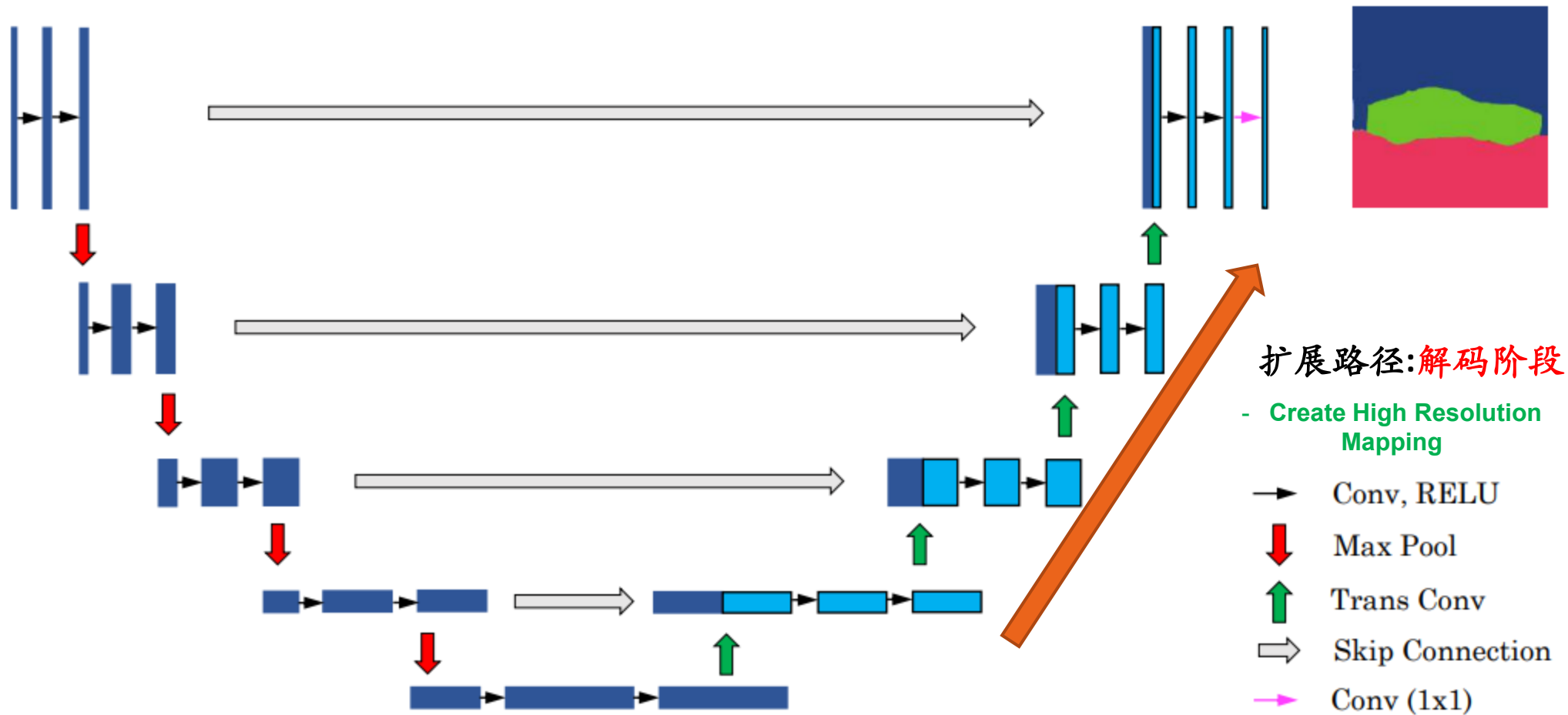
Ronneberger et al. (2015) U-net Architecture

U-Net 架构



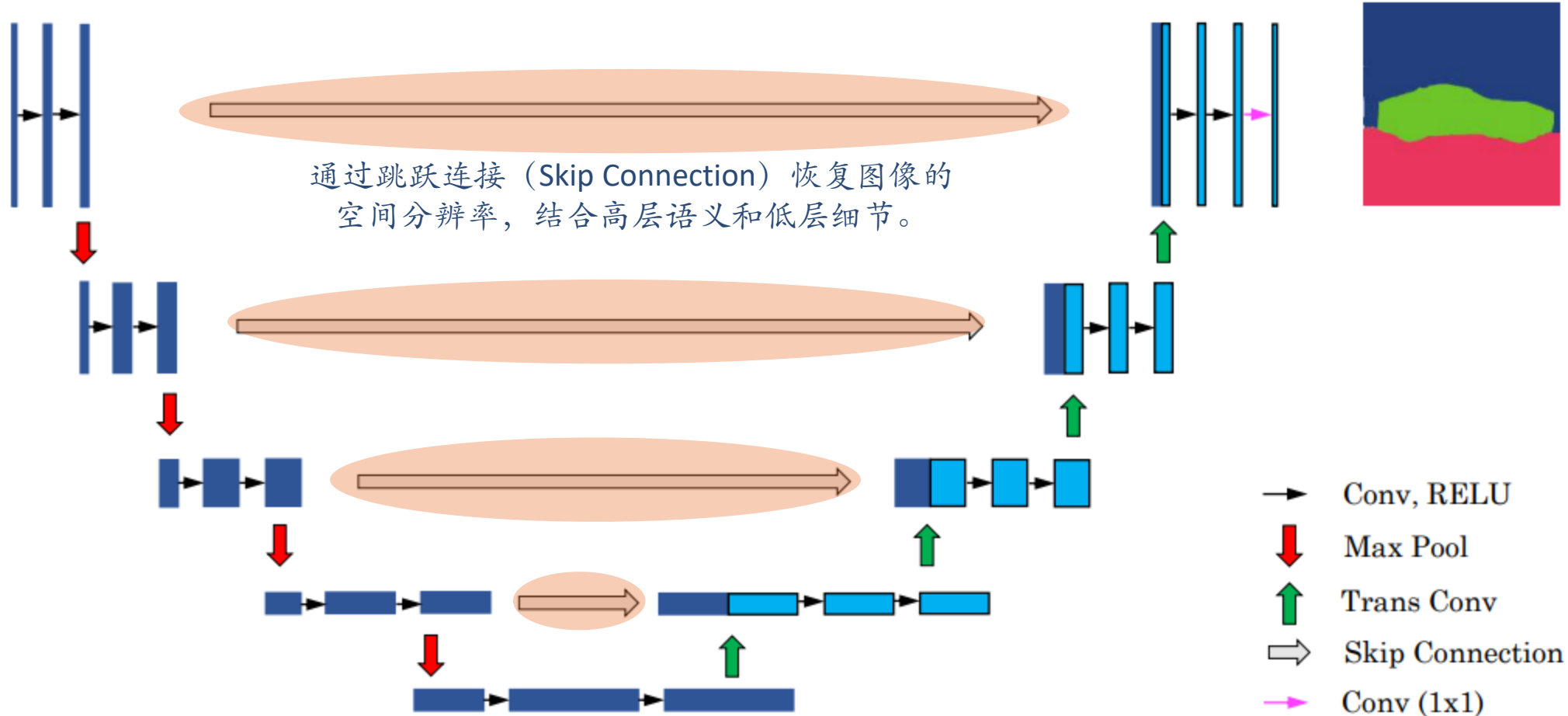
Ronneberger et al. (2015) U-net Architecture

U-Net 架构

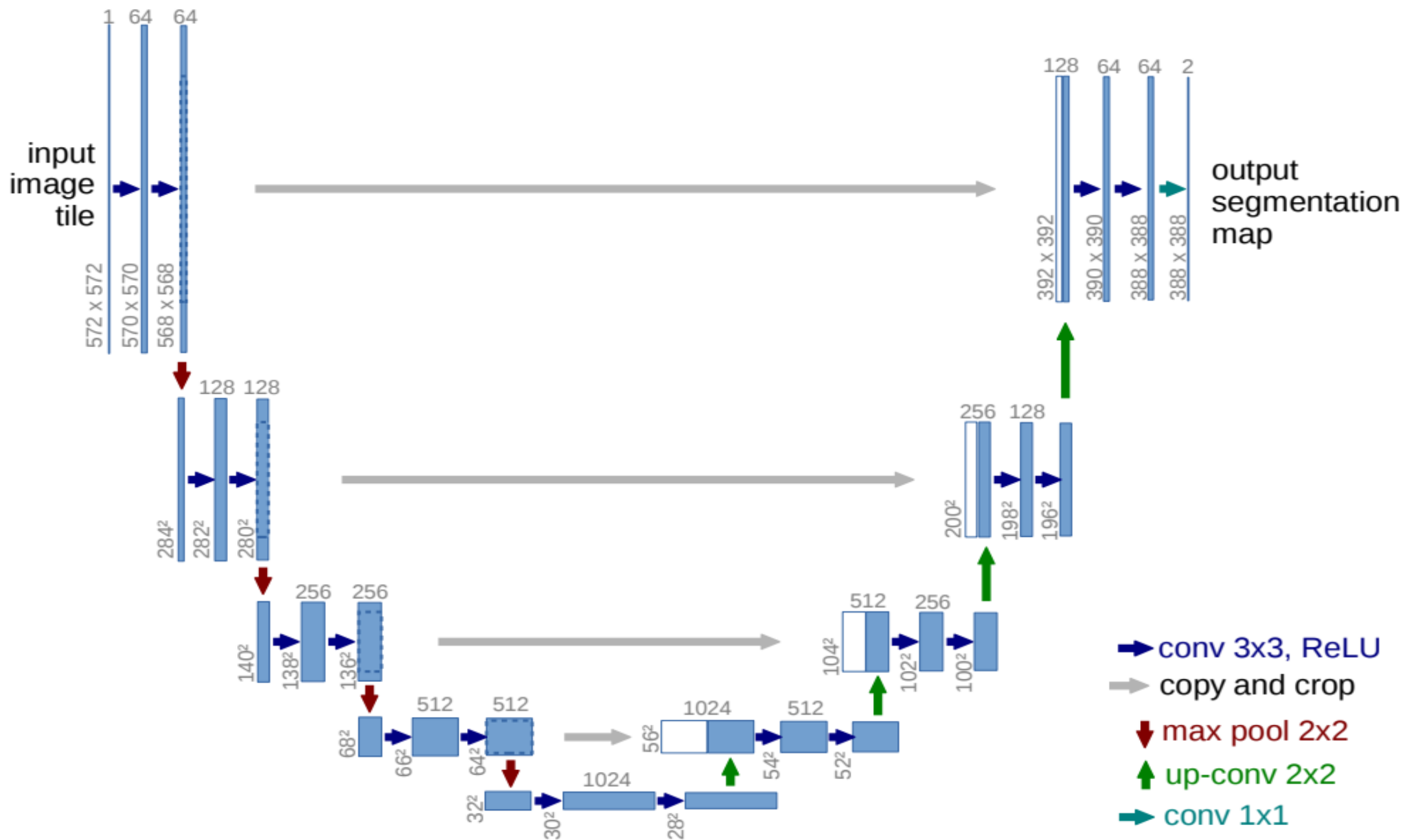


Ronneberger et al. (2015) U-net Architecture

U-Net 架构



Ronneberger et al. (2015) U-net Architecture



HeLa 细胞分割实验

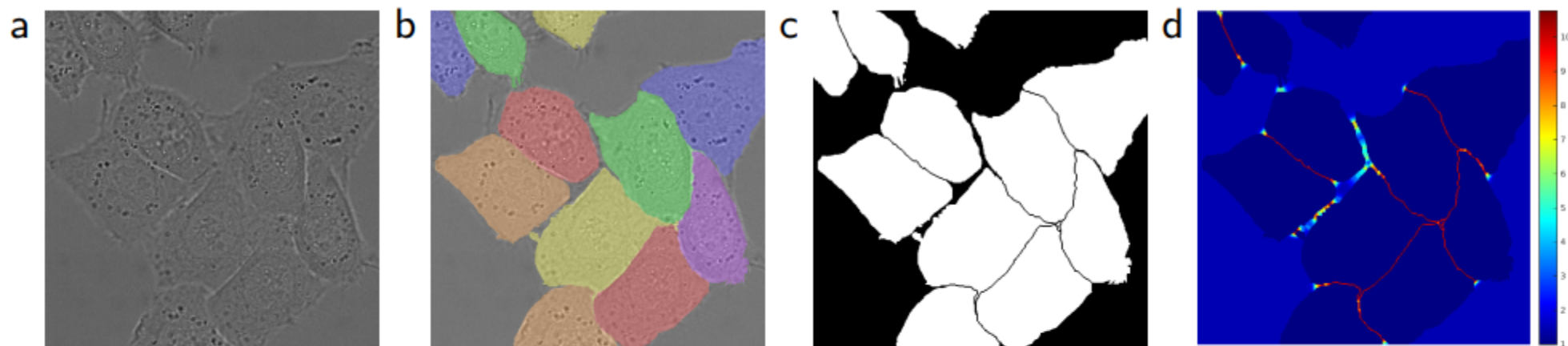


Fig. 3. HeLa cells on glass recorded with DIC (differential interference contrast) microscopy. (a) raw image. (b) overlay with ground truth segmentation. Different colors indicate different instances of the HeLa cells. (c) generated segmentation mask (white: foreground, black: background). (d) map with a pixel-wise loss weight to force the network to learn the border pixels.



概率密度估计

从重构模型到生成模型

▶ 自编码器: $x \rightarrow z \rightarrow \hat{x}$

▶ 表示学习和重构

▶ 生成模型: $z \rightarrow x$

▶ 如何生成新的样本? 需要理解数据分布: $p_{data}(x)$

▶ 完成概率密度估计后, 从数据分布 $p_{data}(x)$ 中生成新的样本

概率密度估计

▶ 参数密度估计 (Parametric Density Estimation)

- ▶ 根据先验知识假设随机变量服从某种分布，然后通过训练样本来估计分布的参数。
- ▶ 估计方法：最大似然估计

$$\log p(\mathcal{D}; \theta) = \sum_{n=1}^N \log p(\mathbf{x}^{(n)}; \theta).$$

▶ 非参数密度估计 (Nonparametric Density Estimation)

- ▶ 不假设数据服从某种分布，通过将样本空间划分为不同的区域并估计每个区域的概率来近似数据的概率密度函数。

参数密度估计

► 多项分布：

数据集 $\mathcal{D} = \{\mathbf{x}^{(n)}\}_{n=1}^N$ 的对数似然函数为

$$\log p(\mathcal{D}|\boldsymbol{\mu}) = \sum_{n=1}^N \sum_{k=1}^K x_k^{(n)} \log(\mu_k). \quad (9.34)$$

多项分布的参数估计为约束优化问题. 引入拉格朗日乘子 λ , 将原问题转换为无约束优化问题.

$$\max_{\boldsymbol{\mu}, \lambda} \sum_{n=1}^N \sum_{k=1}^K x_k^{(n)} \log(\mu_k) + \lambda \left(\sum_{k=1}^K \mu_k - 1 \right). \quad (9.35)$$

分别求上式关于 μ_k, λ 的偏导数, 并令其等于 0. 可得,

$$\mu_k^{ML} = \frac{m_k}{N}, \quad 1 \leq k \leq K \quad (9.36)$$

其中 $m_k = \sum_{n=1}^N x_k^{(n)}$ 为数据集中取值为第 k 个状态的样本数量.

参数密度估计一般存在以下问题

▶ 模型选择问题

- ▶ 如何选择数据分布的密度函数？
- ▶ 实际数据的分布往往是非常复杂的，而不是简单的正态分布或多项分布。

▶ 不可观测变量问题

- ▶ 即我们用来训练的样本只包含部分的可观测变量，还有一些非常关键的变量是无法观测的，这导致我们很难准确估计数据的真实分布。

▶ 维度灾难问题

- ▶ 高维数据的参数估计十分困难
- ▶ 随着维度的增加，估计参数所需要的样本数量指数增加。在样本不足时会出现过拟合。



非参数密度估计

非参数密度估计

- ▶ 对于高维空间中的一个随机向量 $\mathbf{x}$ ，假设其服从一个未知分布 $p(\mathbf{x})$ ，则 $\mathbf{x}$ 落入空间中的小区域 $\mathcal{R}$ 的概率为

$$P(\mathbf{x} \in \mathcal{R}) = \int_{\mathcal{R}} p(\mathbf{x}) d\mathbf{x}.$$

- ▶ 给定 N 个训练样本 $D = \{\mathbf{x}^{(n)}\}_{n=1}^N$ ，落入区域 $\mathcal{R}$ 的样本数量 K 服从二项分布

$$P_K = \binom{N}{K} P^K (1-P)^{1-K},$$

- ▶ 假设区域 $\mathcal{R}$ 足够小，且其中密度近似常数，则：

$$P(\mathbf{x} \in \mathcal{R}) \approx p(\mathbf{x})V.$$

- ▶ 另一方面，若 N 个样本中有 K 个落入 $\mathcal{R}$ ，则：

$$P(\mathbf{x} \in \mathcal{R}) \approx \frac{K}{N}.$$

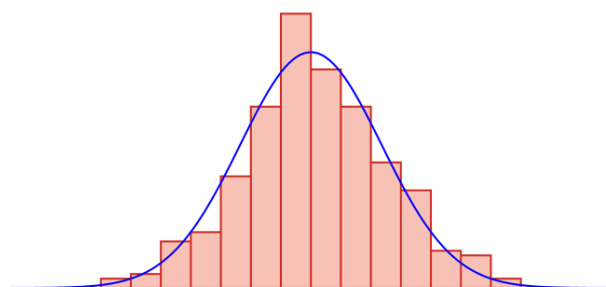
- ▶ 结合上述两个公式，得到非参数密度估计的基本公式

$$p(\mathbf{x}) \approx \frac{K}{NV}$$

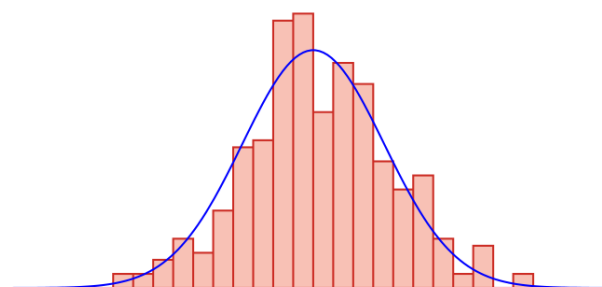
直方图方法 (Histogram Method)

- ▶ 一种非常直观的估计连续变量密度函数的方法，可以表示为一种柱状图。

以一维随机变量为例，首先将其取值范围分成 M 个连续的、不重叠的区间 (bin), 每个区间的宽度为 Δ_m . 给定 N 个训练样本 $\mathcal{D} = \{x^{(n)}\}_{n=1}^N$, 我们统计这些样本落入每个区间的数量 K_m , 然后将它们归一化为密度函数.



(a) 10 个区间 (bin)

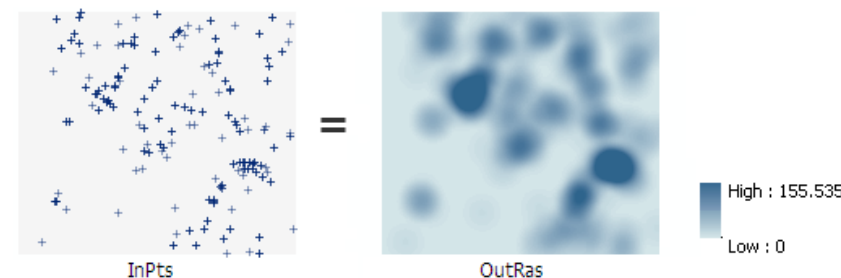


(b) 30 个区间 (bin)

核密度估计 (Kernel Density Estimation)

►核密度估计是一种直方图方法的改进。

►也叫Parzen窗方法



►假设 $\mathcal{R}$ 为d维空间中的一个以点 $\mathbf{x}$ 为中心的“超立方体”，并定义核函数来表示一个样本 $\mathbf{z}$ 是否落入该超立方体中

$$\phi\left(\frac{\mathbf{z} - \mathbf{x}}{h}\right) = \begin{cases} 1 & \text{if } |z_i - x_i| < \frac{H}{2}, 1 \leq i \leq D \\ 0 & \text{else} \end{cases}$$

►点 $\mathbf{x}$ 的密度估计为
$$p(\mathbf{x}) = \frac{K}{NH^D} = \frac{1}{NH^D} \sum_{n=1}^N \phi\left(\frac{\mathbf{x}^{(n)} - \mathbf{x}}{H}\right).$$

K近邻方法

- ▶ 核密度估计方法中的核宽度是固定的，因此同一个宽度可能对高密度的区域过大，而对低密度区域过小。
- ▶ 一种更灵活的方式是设置一种可变宽度的区域，并使得落入每个区域中样本数量为固定的 K 。
- ▶ 要估计点 x 的密度，首先找到一个以 x 为中心的球体，使得落入球体的样本数量为 K ，就可以计算出点 x 的密度：

$$\hat{p}(x) = \frac{K}{NV(x)},$$

- ▶ 其中 $V(x)$ 是包含 K 个近邻所需球体的体积。高密度区域 $V(x)$ 小，低密度区域 $V(x)$ 大。

非参数密度估计

▶ 非参数密度估计方法：

- ▶ 例如 KDE 和 KNN 密度估计，
- ▶ 在预测或估计新点密度时通常需要依赖训练样本，
- ▶ 因此存储和计算成本较高。

▶ 参数密度估计方法：

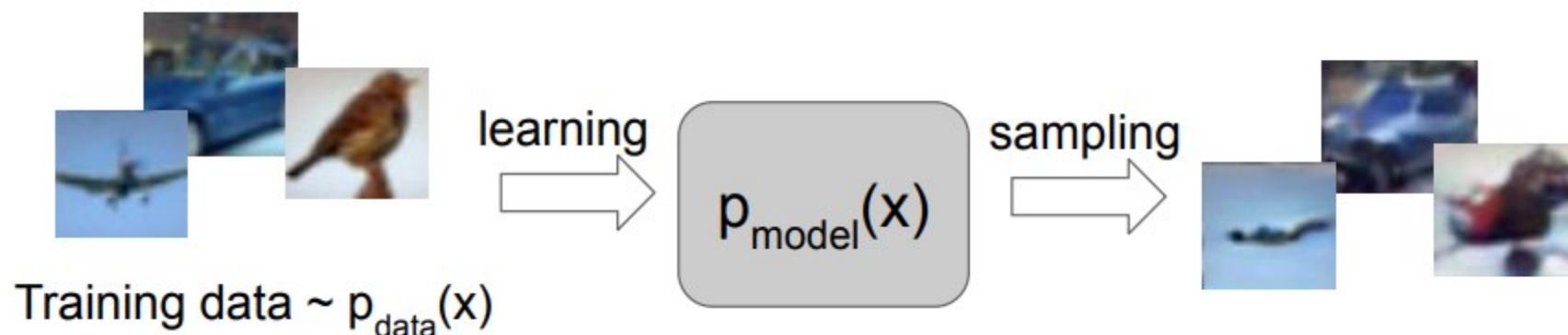
- ▶ 不需要保留整个训练集，因此在存储和计算上更加高效。



生成模型

生成模型

- ▶ 给定训练数据，从同样的分布中生成新的样本



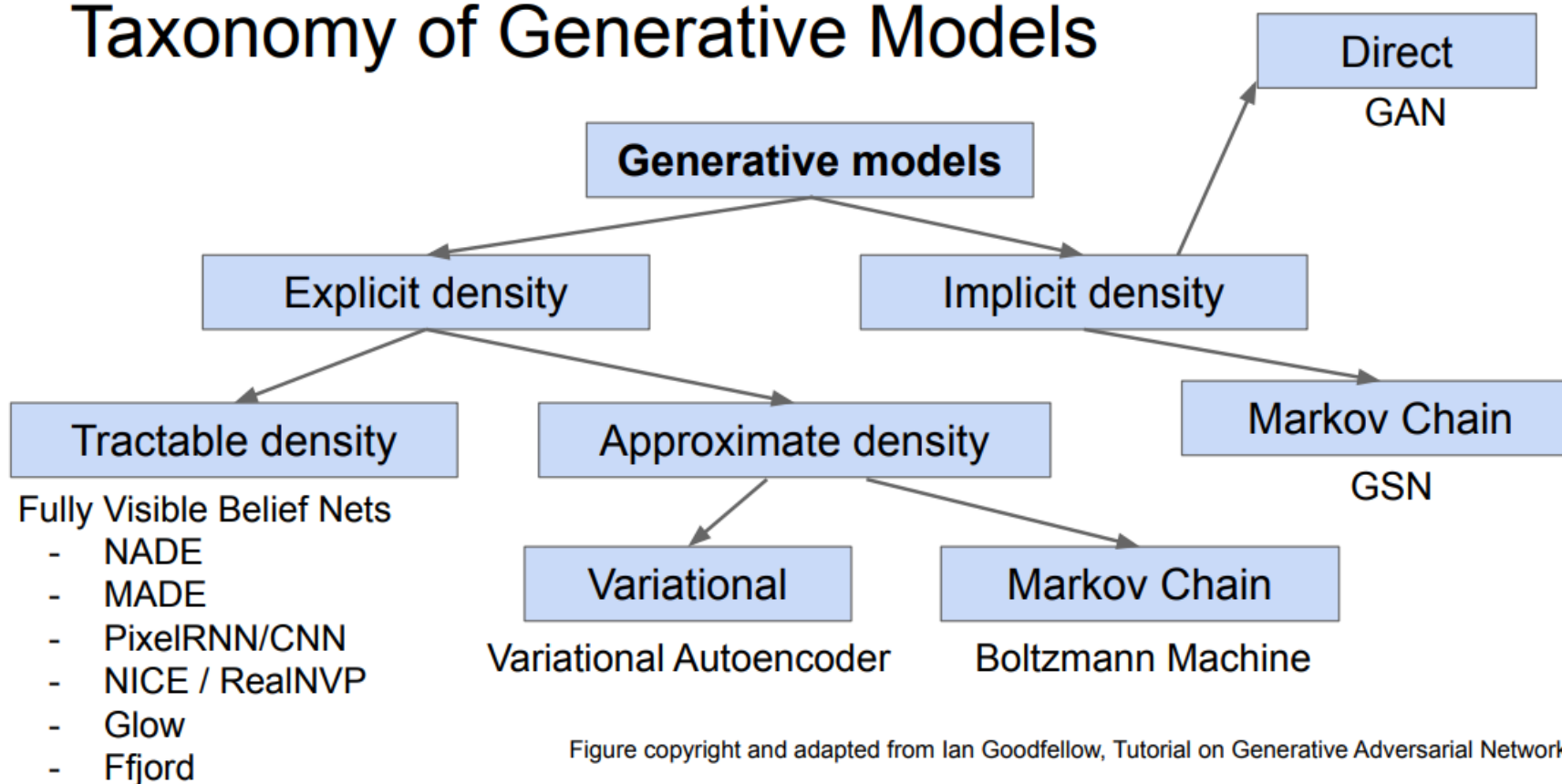
- ▶ 目标:

- ▶ 学习 $p_{model}(x)$ 以近似真实的分布 $p_{data}(x)$
- ▶ 再根据 $p_{model}(x)$ 生成新的样本 x

概率密度估计

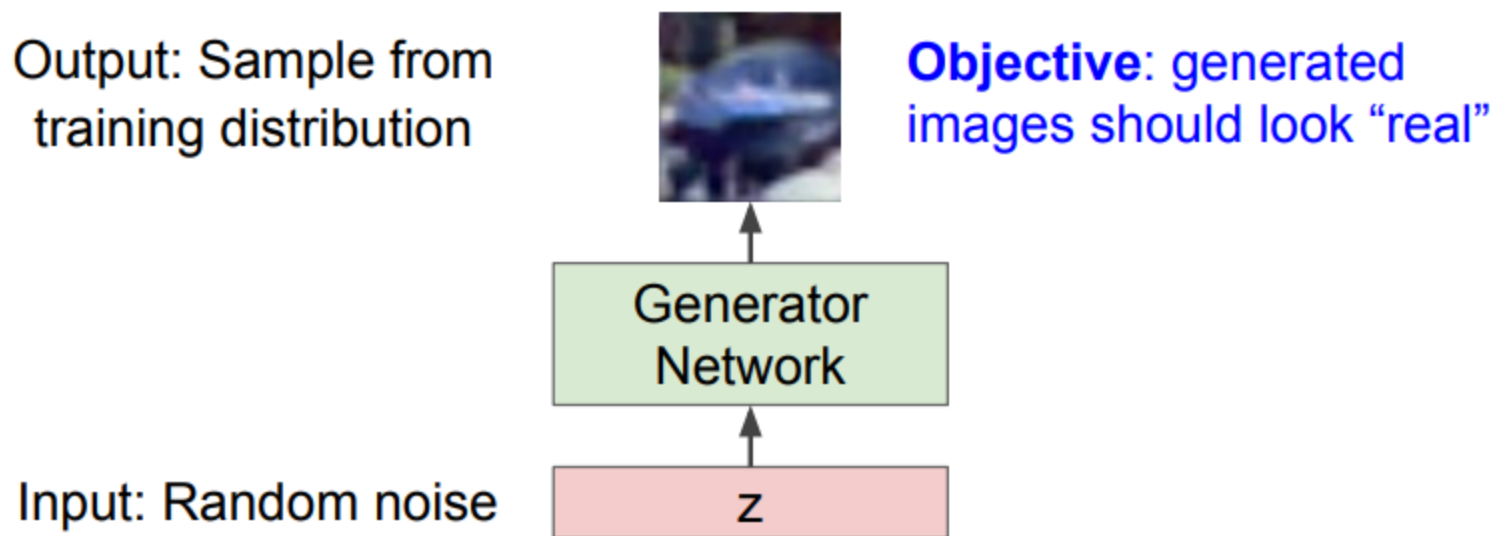
- ▶ 显式的密度估计：显式的定义并求解出 $p_{\text{model}}(x)$
- ▶ 间接的密度估计：不需要显式定义 $p_{\text{model}}(x)$

Taxonomy of Generative Models



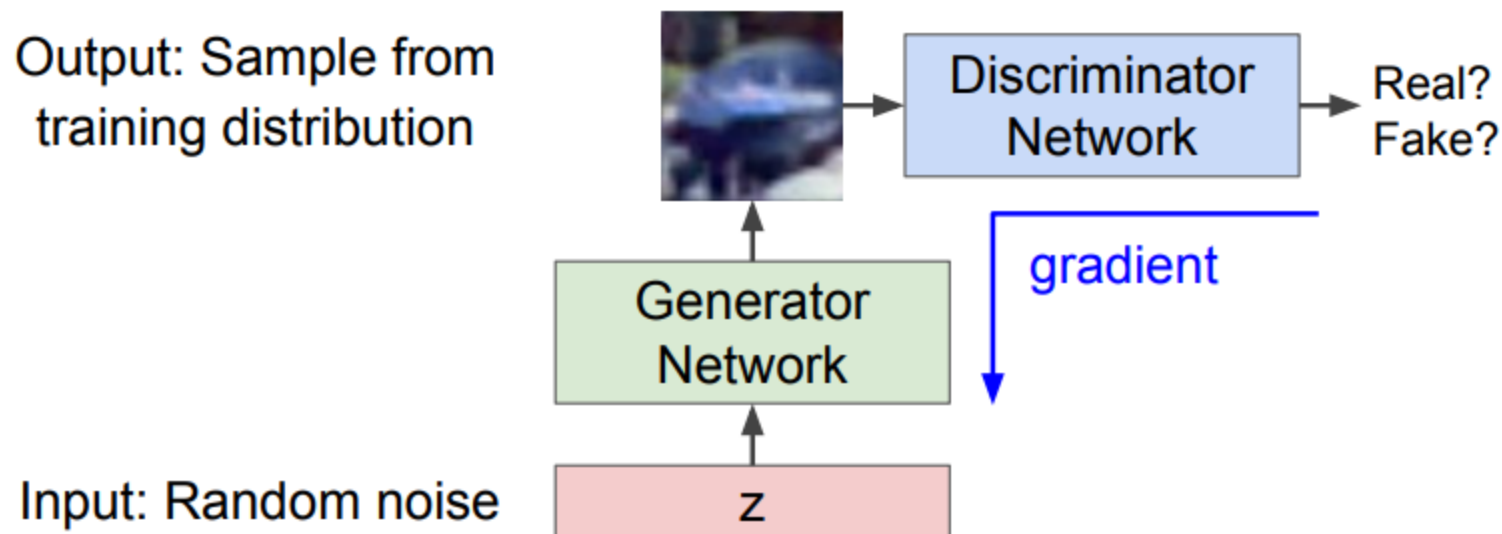
Generative Adversarial Networks (GANs)

- ▶ **如何不用显式的模型密度函数估计，而是直接生成样本？**
- ▶ 问题：需要从复杂的高维训练分布中采样，但无法直接实现。
- ▶ 方案：先从易于采样的简单分布（如随机噪声）中取样，再学习一种变换方式，将其映射为训练分布。



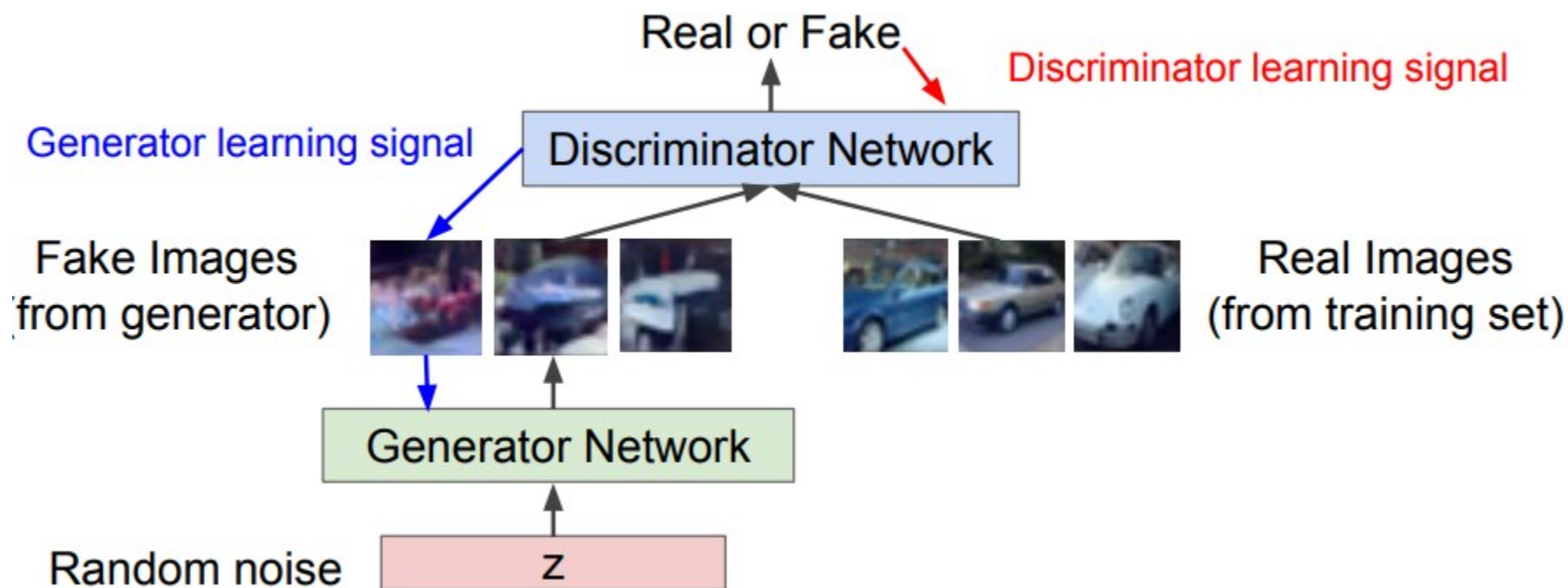
Generative Adversarial Networks (GANs)

- ▶ 如何不用显式的模型密度函数估计，而是直接生成样本？
- ▶ 问题：我们无法确定隐变量样本 z 对应哪张训练图像，因此不能通过重构训练图像来完成学习。
- ▶ 方案：引入判别器网络，判断生成的图像是否属于数据分布（即是否为真实图像）。



训练GAN网络: 双人博弈

- ▶生成网络：生成足以以假乱真的图像，欺骗判别器。
- ▶判别器网络：区分真实图像与伪造图像。



训练GAN网络: 双人博弈

- ▶ 生成网络: 生成足以以假乱真的图像, 欺骗判别器。
- ▶ 判别器网络: 区分真实图像与伪造图像。

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log \underbrace{D_{\theta_d}(x)}_{\text{Discriminator output for real data } x} + \mathbb{E}_{z \sim p(z)} \log(1 - \underbrace{D_{\theta_d}(G_{\theta_g}(z))}_{\text{Discriminator output for generated fake data } G(z)}) \right]$$

Discriminator outputs likelihood in (0,1) of real image

Generator objective

Discriminator objective

- ▶ 同时在minimax game中进行训练

训练GAN网络: 双人博弈

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

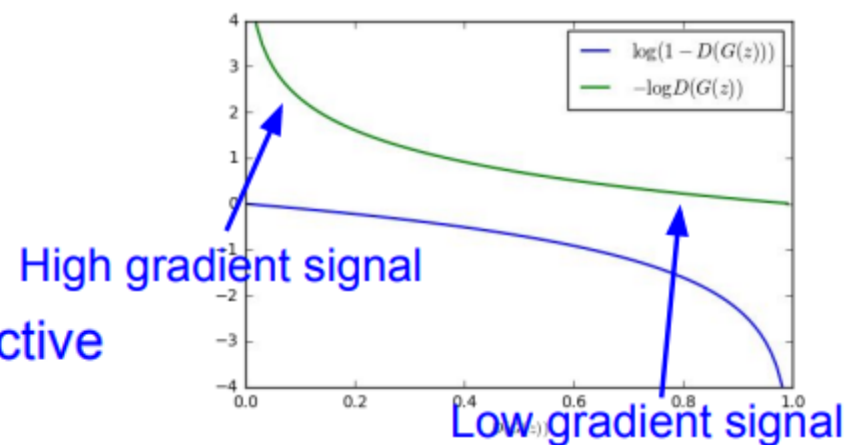
$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Gradient descent** on generator

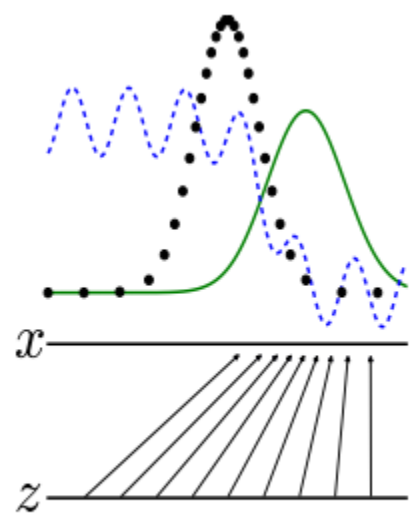
$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

2. **Instead: Gradient ascent** on generator, **different objective**

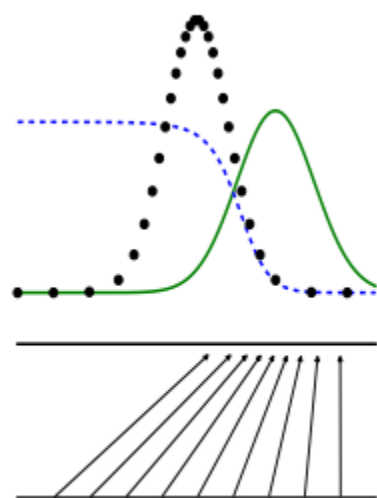
$$\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$$



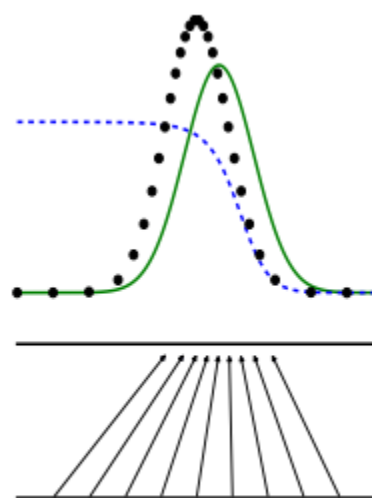
训练GAN网络: 双人博弈



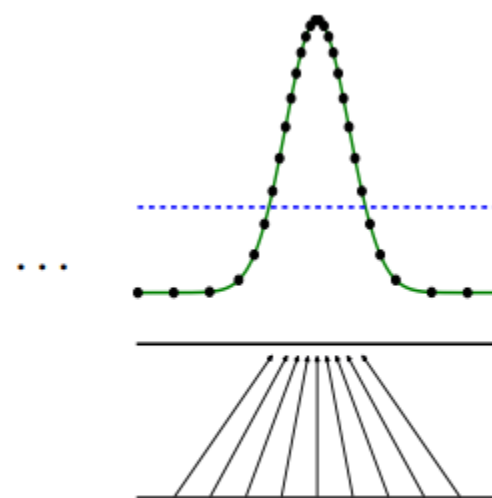
(a)



(b)



(c)



(d)

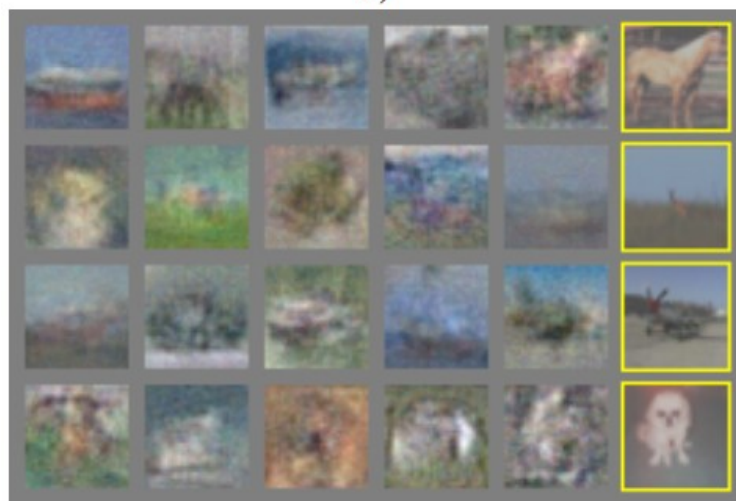
GAN网络：生成效果



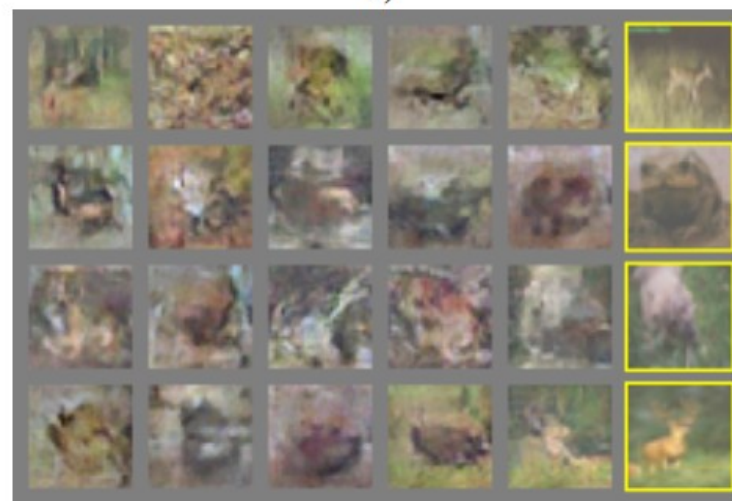
a)



b)



c)



d)